Use Cases

1. **Load textures:** the developer loads base or custom textures for towers, enemies, strategic assets, GUI, background (map) and projectiles.
2. **Remove a texture:** the developer removes a texture
3. **Level entities creation:** Level entities creation: the developer creates entities (e.g. tower spots, enemy types) by selecting the entity type.
4. **Configure wave entity:** the developer sets spawn rate, types of enemy entities for each enemy path involved in wave, set trigger condition
5. **Handle error:** the editor displays an error message.
6. **Get confirmation:** the editor asks for developers' confirmation before performing some action.
7. **Damage type configuration:** the developer can add new types of damage (instant, ongoing, from some effects, etc.).
8. **Tower type configuration:** the developer sets the damage type, attack range, attack cooldown, upgrade/sell system, active/inactive modes of tower entities.
9. **Enemy type configuration:** the developer sets enemy speed, HPs amount and texture.
10. **Grid and map layout setup**: configuring tile-based grid properties, background assets and zones of the level map.
11. **Enemy path configuration:** defining and drawing the route trajectories and movement waypoints that enemies will follow. Setting reference to the HP counter, associated with this path
12. **Entity placement and spawning**: selecting game entities (towers, enemy spots, strategic assets) from a list and placing them onto the map.

13. **Entity duplication:** cloning an existing entity along with all its configured fields.
14. **Group operations**: selecting multiple entities to move, duplicate or delete them in group.
15. **Export game:** the developer chooses the directory to which he wants to export his game and the editor exports the game there by:
    a. Creating game files structure
    b. Exporting entities
    c. Generating level configuration
    d. Creating game executable
16. **Import game:** the developer imports previously exported game so he can edit it inside of the editor.
17. **Launch the game:** the user launches the game using the generated executable.
18. **Place a tower:** the player selects a tower type and places it in an available position on the map.
19. **Sell a tower:** the player sells a selected tower and receives part of its cost back.
20. **Upgrade a tower:** the player upgrades a selected tower using available resources.
21. **Use an ability:** the player activates an available special ability during the game.
22. **Start the next wave:** the player starts the next enemy wave.
23. **Change game speed:** the player increases or decreases the game speed

Detailed Descriptions

1. Load texture

Actors: developer
Goal: load base or custom texture for towers, enemies, strategic assets, GUI elements, map background or projectiles.
Trigger condition: developer opens textures/assets panel in the editor.
Precondition/Context: The editor is running, a level is opened.
Extensions:

- (step 3, the selected file has an unsupported format, **Handle error** with appropriate error message)
- (step 3, a texture with the same identifier or name already exists, **Handle error** with appropriate error message)
- (step 3, file system error, **Handle error** with appropriate error message)
- (step 4, the texture file is in unsupported format, **Handle error** with appropriate error message)

Main success scenario:

1. Developer selects a texture category.
2. Developer imports a base or custom texture file and chooses the texture name.
3. Editor validates the file and assigns the texture to the selected category.
4. Editor previews the texture.
5. Developer saves the changes.
6. Editor writes the texture reference into the in-memory level descriptor.

2. Remove a texture

Actors: developer
Goal: remove a texture.
Trigger condition: developer opens textures/assets panel in the editor and presses “remove” button by one of the textures name
Precondition/Context: The editor is running, a level is opened.
Main success scenario:

1. The editor removes all references to this texture from all game entities
2. The editor removes the texture reference from the in-memory level descriptor

3. Level entities creation

Actors: developer

Goal: create level entities

Trigger condition: developer press a button “new entity” in the editor menu

Precondition/Context: editor is running, a level is opened.

Main success scenario:

1. Developer selects the entity type (e.g. tower type, enemy type, tower spot).
2. The editor add new entity to the list of entities

4. Configure wave entity

Actors: developer

Goal: configure enemy waves by setting spawn rate, enemy entity types for each enemy path, and trigger condition.

Precondition/Context: editor is running, a level is opened, enemy paths are available, enemy entity types are available.

Main success scenario:

1. Developer opens one of wave entities' editing menu
2. Developer adds an enemy path to list of paths involved in wave
3. Developer sets the enemy entity types for the added path
4. Developer sets the spawn rate for the added path
5. If developer wants to add one more enemy path to the wave, go to step 2
6. Developer sets the trigger condition for the wave
7. Developer saves the wave configuration

5. Handle error

Actors: developer

Goal: get the information about an error occurred during editing or configuration.

Precondition/Context: editor is running, the developer is performing an erroneous operation in the editor.

Main success scenario:

1. The editor creates a new window with:
    a. The error message displayed
    b. The "OK" button
2. The developer presses the "OK" button and the window closes

6. Get confirmation

Actors: developer

Goal: insure correctness of the operation that will be performed be the editor and possibly cancel it

Main success scenario:

1. The editor creates a new window with:
    a. The info about consequences of the operation that will be performed
    b. The “Proceed” button
    c. The “Cancel” button
2. If the developer presses “Proceed” button, the window closes and editor proceeds with the operation
3. Else, the window closes and editor doesn’t proceed with the operation

7. Damage type configuration

Actors: developer

Goal: configure a type of damage used by game entities.

Precondition/Context: editor is running, a level or project is opened.

Main success scenario:

1. Developer opens one of damage type entities’ editing menu
2. Developer configures the damage behavior by changing damage entity fields’ values
3. Developer saves the damage type configuration

8. Tower type configuration

Actors: developer

Goal: configure a tower type that can be placed by player

Precondition/Context: the editor is running, textures for the tower and its projectiles have been loaded.
Main success scenario:

1. Developer opens one of tower type entities' editing menu
2. The developer assigns texture for the tower body and selects projectiles for firing;
3. The developer configures target logic (first, last, strongest, weakest, closest);
4. The developer configures attack range and cooldown;
5. The developer add references to tower type entities to which this tower can be promoted
6. The developer previews the tower in context (range circle, UI card);
7. Developer saves the tower type configuration

9. Enemy type configuration

Actors: developer
Goal: configure an enemy type that can be spawned during wave
Precondition/Context: the editor is running, textures for the enemy have been loaded.
Main success scenario:

1. Developer opens one of enemy type entities' editing menu
2. The developer assigns texture for the enemy body
3. The developer configures enemy speed
4. The developer configures enemy HPs amount
5. Developer saves the enemy type configuration

10. Grid and map layout setup

Actors: developer

Goal: configure the tile-based grid properties, background assets, and zones of the level map.
Trigger condition: the developer opens the map/grid settings panel or enters level editing mode.
Precondition/Context: the editor is running, a project or level is opened or created, background textures/tilesets have been loaded (or defaults are available).
Extensions:

- (step 3, a referenced background asset is missing, **Handle error** with appropriate message)

Main success scenario:

1. The developer opens the grid and map layout settings panel;
2. The developer configures grid properties (cell size, map width/height in cells, origin);
3. The developer assigns background assets (tileset or background image) to the map;
4. The developer defines zones of the map (buildable, for path, decorative) with their boundaries and properties;
5. The developer previews the map layout in the editor viewport;
6. The developer saves; the editor writes grid, background, and zone data into the in-memory level descriptor.

Alternative scenarios:

- The developer changes cell size - the editor rescales or re-snaps placed entities;
- The developer cancels changes — the map layout remains unchanged.

11. Enemy path configuration

Actors: developer

Goal: define and draw route trajectories with waypoints that enemies follow from spawn points to the strategic asset.
Precondition/Context: the editor is running, a level is opened, map layout is set up
Extensions:

- If the path breaks on empty/unbuildable tiles or does not reach the strategic asset - Handle error;

Main success scenario:

1. Developer opens one of enemy path entities' editing menu and presses "draw a path" button and enters path editing mode
2. The developer places sequential waypoints on the grid to define the movement trajectory
3. When the arrangement is complete, the developer clicks the "complete editing" button and exits the special editor mode.
4. The developer set reference to the HP counter entity, which will be decreased by enemies reaching the end of this path
5. Developer saves the enemy path configuration

Alternative scenarios:

- The developer moves a waypoint - the connection to neighboring waypoints shifts, but is maintained;
- The developer deletes a waypoint - neighboring ones are connected to each other;

12. Entity placement and spawning

Actors: developer
Goal: select placeable game entities (tower spots, ui elements) from a list of entities and place them onto the map.

Trigger condition: the developer opens the entity list in the level editor.
Precondition/Context: the editor is running, a level is opened, entities have been created, the map layout (grid, zones) is set up.
Extensions:

- If an entity is placed on an invalid tile - Handle error;

Main success scenario:

1. The developer selects an entity;
2. The developer moves the cursor over the map and places the entity on a valid tile;
3. Developer saves the entity configuration

13. Entity duplication

Actors: developer
Goal: clone an existing entity along with all its configured components.
Trigger condition: the developer invokes “duplicate” on a selected entity.
Precondition/Context: the editor is running, a level is opened, an entity is selected in the editor.
Main success scenario:

1. The developer selects an entity on the map or the entity list;
2. The developer invokes “duplicate”;
3. Editor creates a copy of the entity with all components;
4. Editor places the clone at an offset position (or prompts the developer to place it);
5. The developer adjusts the clone’s position if needed;
6. The developer saves; the editor serialized the new entity into the level configuration.

Alternative scenarios:

- The developer cancels placement of the clone - the clone is discarded.

14. Group operations

Actors: developer
Goal: select multiple entities to move, duplicate or delete them.
Trigger condition: the developer selects multiple entities and invokes a group action.
Precondition/Context: the editor is running, a level is opened, multiple entities exist on the map.
Extensions:

- If a move would place any entity on an invalid tile or overlap - Handle error;
- If the selection is empty - Handle error.

Main success scenario:

1. The developer selects multiple entities;
2. The developer invokes a group action: move, duplicate or delete;
3. The developer saves; the editor serializes the updated set of entities into the level configuration.

Alternative scenarios:

- The developer cancels the group operation - the editor reverts to the pre-action state.

15. Export game

Actors: developer
Goal: export game to some directory, so it can be distributed to players, launched or imported back to the same game editor.
Trigger condition: the developer presses the “export” button in the editor menu.
Precondition/Context: the editor is running, a level is opened
Extensions:

- (step 3, see condition in step 3, **Get confirmation**)
- (steps 3 and 7-10, file system error, **Handle error** with appropriate message)

Main success scenario:

1. The editor creates a window, there developer can choose the export directory
2. The developer choose some export directory
3. If the export directory isn't empty, call "Get confirmation" use case to ask for confirmation to clear the export directory before export
4. Else, got to step 7
5. If the developer cancel the operation, go to the step 2 and wait for another export directory chosen
6. Else, clear the export directory and go to step 7
7. The editor closes the window created on step 1
8. The editor creates appropriate directories inside the export directory
9. The editor serializes all game entities to file/files inside of the appropriate subdirectory of the export directory. These files have some format recognizable by both the editor and the game engine executable
10. The editor generates a level description as a configuration file of the format recognizable by both the editor and the game engine executable
11. The editor creates an executable shell script in the export directory. This script executes the game engine executable and passes the export directory as an argument

16. Import game

Actors: developer
Goal: edit previously exported game in the editor

Trigger condition: the developer presses the “import” button in the editor menu.
Precondition/Context: the editor is running.
Extensions:

- (step 3, if the export directory or any file in it has invalid structure, **Handle error** with appropriate message)
- (step 3, file system error, **Handle error** with appropriate message)

Main success scenario:

1. The editor creates a window, there developer can choose the directory from which to import the game
2. The developer choose some directory
3. The editor reads all the files in export directory and opens the project for editing

17. Launch the game

Actors: player
Goal: launch the game
Trigger condition: the player executes executable shell script exported by the game editor
Precondition/Context: the player has the game files exported by the game editor
Main success scenario:

1. The shell script executes the game engine executable searched in the PATH environmental variable and provides the directory there this shell script resides (the export directory) as an only argument

18. Place a tower

Actors: player
Goal: select a tower type and place it in an available position on the map to defend against enemies.

Trigger condition: player opens the tower selection UI
Precondition/Context: game is running; player has sufficient resources (gold/mana); the selected tower type is unlocked; the map grid is visible; game is not on pause
Extensions:

- Handle error - game displays a visual/audio error indicator, such as a red ghost tower or an error tooltip

Main success scenario:

1. Player selects a tower type from the UI palette;
2. Player moves the cursor over an available position on the map grid;
3. Player clicks the left mouse button to confirm placement.

Alternative scenarios:

- Player drags the tower icon from the UI palette and drops it directly onto an available position on the map;

19. Sell a tower

Actors: player
Goal: remove a placed tower from the map and receive a portion of its cost back.
Trigger condition: player opens the tower sell UI
Precondition/Context: game is running; at least one tower is placed on the map; game is not on pause
Extensions:

- Handle error - game displays an error message/tooltip explaining why the tower cannot be sold

Main success scenario:

1. Player clicks on a placed tower to select it;
2. Player clicks the "Sell" button in the tower's info panel.

3. Player gets money

Alternative scenarios:

- Player selects the tower and presses a designated "Sell" hotkey (e.g., 'X');

20. Upgrade a tower

Actors: player
Goal: improve a selected tower's stats or unlock new abilities using available resources.
Trigger condition: player opens the tower upgrade UI
Precondition/Context: game is running; the selected tower has available upgrade tiers; player has sufficient resources; game is not on pause
Extensions:

- Handle error - game displays an error message/tooltip with the specific requirements, button remains grayed out

Main success scenario:

1. Player clicks on a placed tower to select it;
2. Player reviews the available upgrade options in the tower's UI panel;
3. Player clicks the specific active "Upgrade" button corresponding to the desired path/option.
4. The tower changes its' type

Alternative scenarios:

- player selects the tower and presses a specific hotkey assigned to the desired upgrade path (e.g., 'U');

21. Use an ability

Actors: player

Goal: activate a special global or targeted ability to destroy enemies
Trigger condition: player opens the abilities UI
Precondition/Context: game is running; the ability is unlocked; the ability is not on cooldown; game is not on pause
Extensions:

- Handle error - game displays an error message, shows cooldown timer overlay, or plays an error sound

Main success scenario:

1. Player clicks on the ability icon in the UI;
2. (If the ability requires targeting) Player clicks on the target location on the map to confirm activation.
3. Ability activates

Alternative scenarios:

- Player presses the designated hotkey for the ability (and uses keyboard/mouse to select the target if required);

22. Start the next wave

Actors: player
Goal: initiate the next sequence of enemy spawns and get extra resources
Trigger condition: player clicks the “Call Next Wave” button
Precondition/Context: game is running; the first wave of enemies was released; game is not on pause
Main success scenario:

1. Player clicks the "Start Next Wave" button in the UI.
2. The next wave starts

Alternative scenarios:

- player presses the designated hotkey for starting the next wave (e.g., Spacebar or 'N');

23. Change game speed

Actors: player
Goal: increase or decrease the simulation speed of the game.
Trigger condition: Player clicks the speed cycle button to rotate through speed tiers (x1->x2->x3)
Precondition/Context: game is running; game is not on pause
Extensions:

- Handle error - game displays an error message/tooltip, speed controls remain disabled/grayed out

Main success scenario:

1. Player clicks the desired speed multiplier button (e.g., 2x, 3x) in the UI.
2. The game speed multiplier changes to chosen value

Alternative scenarios:

- player presses the designated hotkeys to cycle through or set the game speed (e.g., '+', '-', or '1', '2', '3');
- player scrolls the mouse wheel while hovering over the speed UI element to change the speed.